

ISO/IEC 27001:2022

Controles 8.25, 8.28 y 8.29

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I

La norma **ISO/IEC 27001:2022** establece los requisitos para un **Sistema de Gestión de Seguridad de la Información (SGSI)**. Es la referencia mundial para proteger confidencialidad, integridad y disponibilidad.

Anexo A — Controles tecnológicos:

- 93 controles en 4 temas: organizacionales, personas, físicos y tecnológicos
- **A.8:** Controles tecnológicos (34 controles)
- Los más relevantes para desarrollo: **8.25, 8.28 y 8.29**
- Forman un cluster que cubre todo el ciclo de vida del desarrollo seguro

SGSI

Sistema de Gestión de Seguridad de la
Información

El control **8.25** establece las reglas generales. El **8.28** se enfoca en el código. El **8.29** verifica antes de producción. Juntos cubren desde planificación hasta mantenimiento.

8.25

Secure Development Life Cycle

Reglas para desarrollo seguro durante todo el ciclo de vida: requisitos, diseño, desarrollo, pruebas, despliegue y mantenimiento.
Separación de entornos y competencia del equipo.

8.28

Secure Coding

Principios de codificación segura: validación de entradas, manejo de secretos, mínimo privilegio, dependencias seguras y manejo de errores. Aplica a código interno y externo.

8.29

Security Testing

Pruebas de seguridad durante desarrollo y antes de producción. SAST, DAST, revisiones de código, escaneo de vulnerabilidades y acceptance testing con criterios claros.

Un **Secure SDLC** integra seguridad en cada fase del desarrollo. Concepto clave: **Shift Left** — mover seguridad hacia fases iniciales, donde es más barato corregir.

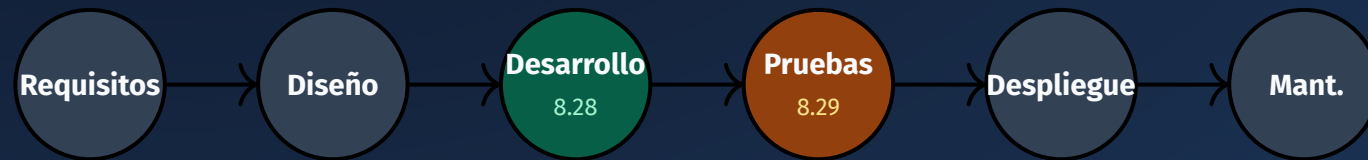
Modelo tradicional

- La seguridad se revisaba **al final** del proyecto
- Vulnerabilidades detectadas tarde
- Costo de corrección alto
- Más componentes afectados

SSDLC — Shift Left

- Seguridad desde **Requisitos**, no al final
- Threat modeling en diseño
- Codificación segura en desarrollo
- Pruebas automáticas en CI/CD
- Hardening en despliegue

El SSDLC integra seguridad en 6 fases. Las fases 3 y 4 se vinculan con los controles **8.28** y **8.29**.



Todos bajo el control **8.25** — Reglas del ciclo de vida seguro.

Fase 1 — Requisitos de Seguridad

Activos: Bases de datos, credenciales, código fuente, registros de auditoría.

Clasificación: Pública, Interna, Confidencial, Restringida.

Requisitos CIA + Trazabilidad:

- **Confidencialidad:** solo autorizados acceden
- **Integridad:** datos no modificados sin autorización
- **Disponibilidad:** servicio operativo
- **Trazabilidad:** acciones auditables

Fase 2 — Diseño Seguro

Threat Modeling (STRIDE): Identificar amenazas en cada componente.

Diagrama de Flujo de Datos:

- Procesos que transforman datos
- Flujos entre componentes
- Almacenes de datos
- Entidades externas

Límites de confianza: Cada frontera requiere controles adicionales.

Fase 3 — Desarrollo (8.28)

Codificación siguiendo los 7 principios.
Revisiones por pares. Herramientas SAST integradas en el IDE.

Fase 4 — Pruebas (8.29)

Pruebas SAST y DAST. Penetration tests.
Criterios: 0 vuln. críticas, cobertura SAST $\geq$ 80%. Si no cumple, no se despliega.

Fase 5 — Despliegue

Hardening: imágenes mínimas, usuarios no root, puertos restringidos. Secretos con variables de entorno. CI/CD con Security Gate.

La metodología **STRIDE** identifica amenazas en cada fase de diseño. Para cada componente se preguntan 6 categorías de amenazas:

Amenaza	Significado	Pregunta clave
Spoofing	Suplantación de identidad	¿Quién es realmente el usuario?
Tampering	Manipulación de datos	¿Pueden alterarse los datos en tránsito?
Repudiation	Negación de acciones	¿Se puede demostrar quién hizo qué?
Info Disclosure	Exposición de información	¿Se puede leer información confidencial?
DoS	Denegación de servicio	¿Se puede interrumpir el servicio?
EoP	Escalada de privilegios	¿Se pueden obtener permisos extra?

El control 8.28 establece principios de codificación segura aprobados. Deben aplicarse a todo el código: desarrollo interno, externo, bibliotecas y dependencias.

1. Nunca confiar en la entrada

Toda entrada es no confiable hasta que se valide. Consultas parametrizadas.

2. Validar siempre

Tipo, longitud, formato, rango antes de procesar.

3. Sanitizar

Preparar datos para uso seguro. Escape HTML para XSS.

4. Mínimo privilegio

Solo permisos necesarios. MongoDB usuario solo para la BD.

5. Secretos

Nunca hardcodear. Usar .env y gestores de secretos.

6. Errores seguros

No exponer stack traces. Mensajes genéricos, logs internos.

7. Dependencias

Escanear, actualizar, eliminar abandonadas.

Refuerzo Revisión por pares + SAST en CI/CD = defensa en profundidad.

El control 8.28 previene las vulnerabilidades del **OWASP Top 10**. Cada categoría se mapea a principios de codificación segura.

Vulnerabilidad	Nivel	Principio de prevención
Injection	Crítica	Validación y sanitización de entradas, consultas parametrizadas
Broken Access Control	Crítica	Control de acceso por roles, mínimo privilegio
Cryptographic Failures	Alta	Cifrado en reposo y tránsito, hashing con bcrypt/argon2
Security Misconfiguration	Alta	Hardening de servidores, contenedores y servicios
Vulnerable Components	Alta	Escaneo de dependencias, actualización periódica
Authentication Failures	Alta	Tokens con expiración, contraseñas hasheadas

El control 8.29 exige pruebas de seguridad durante el desarrollo y antes de producción. Los hallazgos se registran, evalúan, corrigen y re-verifican.

SAST — Análisis Estático

Analiza código sin ejecutarlo. Escáner busca patrones de vulnerabilidades. Detecta fallos temprano. Se integra en CI/CD y en el IDE.

ESLint, Semgrep, SonarQube, Bandit

Code Review

Revisión humana. Encuentra errores de lógica, decisiones de diseño inseguras y patrones que el escáner no reconoce. Obligatoria antes de cada merge.

DAST — Análisis Dinámico

Prueba la app ejecutándose. Scanner envía ataques simulados. Cubre superficie de ataque expuesta.

OWASP ZAP, Burp Suite

Penetration Testing

Especialista compromete el sistema de forma controlada. Demuestra impacto real. Prueba más cercana a un ataque real.

Los controles 8.25, 8.28 y 8.29 se concretan en un pipeline de integración continua seguro. Cada paso aplica un control específico.



El **Security Gate** verifica: 0 vuln. críticas, 0 altas, cobertura SAST $\geq 80\%$, dependencias sin CVEs críticos. Si no se cumplen, el pipeline se detiene.

El acceptance testing es la verificación final antes de producción.

Criterios de aceptación:

- 0 vulnerabilidades críticas
- 0 vulnerabilidades altas
- Cobertura SAST $\geq 80\%$
- Dependencias sin CVEs críticos
- Pruebas ejecutadas y aprobadas
- Configuración endurecida

Si no cumple, no se despliega.

Los hallazgos deben:

1. **Registrarse** en seguimiento
2. **Evaluar** por severidad
3. **Corregirse** antes de avanzar
4. **Re-verificarse** tras corrección

Métricas para evaluar el cumplimiento de los controles 8.25, 8.28 y 8.29 con sus ecuaciones matemáticas.

Métrica	Fórmula	Objetivo
Densidad de vulnerabilidades	$D_v = \frac{V}{\text{KLOC}}$	$D_v < 1$
Cobertura de análisis SAST	$C = \frac{A_{\text{esc}}}{A_{\text{tot}}} \times 100$	$C \geq 80\%$
Tiempo medio de remediación	$T_{\text{rem}} = \frac{\sum \Delta t}{n}$	$T_{\text{rem}} \leq 7 \text{ días}$
Tasa de aceptación	$R = \frac{P_{\text{apr}}}{P_{\text{tot}}} \times 100$	$R = 100\%$
Madurez SSDLC	$M = \frac{\sum P_i w_i}{\sum w_i}$	$\text{Nivel} \geq 2$

Plataforma web full-stack para estudiantes de la UNAM. Foro académico con stack tecnológico estándar.

Stack tecnológico

- **Frontend:** React 18, TypeScript, Vite, Tailwind CSS
- **Backend:** Node.js, Express.js, Mongoose
- **Base de datos:** MongoDB 7
- **Almacenamiento:** MinIO (compatible S3)
- **Infraestructura:** Docker, Docker Compose, Nginx
- **Autenticación:** JWT, bcrypt, roles: student/moderator/admin

Métricas del proyecto

- Densidad vuln.: $D_v = 0.6$ **Bajo**
- Cobertura SAST: $C_{\text{SAST}} = 88.9\%$
- Tiempo remediación: $T_{\text{rem}} = 5$ días
- Tasa aceptación: $R = 100\%$
- Madurez SSDLC: Nivel 2 (Gestionado)

Hallazgos reales de seguridad en la configuración del proyecto. Demuestran por qué el control 8.25 es necesario.

Hallazgo	Ubicación	Riesgo
JWT_SECRET hardcodeado	docker-compose.yml:67	Firma de tokens válidos
Credenciales MinIO por defecto	docker-compose.yml:23-24	Acceso no autorizado a archivos
MongoDB puerto expuesto	docker-compose.yml:7	Acceso directo a la BD

Un auditor no busca una herramienta. Busca un proceso definido, aplicado y demostrable.

Evidencias de 8.25:

- Política de Secure SDLC documentada
- Capacitación del equipo en seguridad
- Requisitos definidos antes del desarrollo
- Threat modeling realizado y documentado
- Control de cambios y pipeline seguro

Evidencias de 8.28 y 8.29:

- Principios de codificación aprobados
- Revisiones de código con hallazgos
- Resultados de SAST y DAST
- Criterios de aceptación documentados
- Pipeline CI/CD con Security Gate

¿Preguntas?

Equipo 5 — IS-722 Calidad de Software

Universidad Nacional de Moquegua — 2026-I